KLH

BACHUPALLY

KLH (Deemed to be University) | Koneru Lakshmaiah Education Foundation | Department of CSE

Student Council Election Tally System

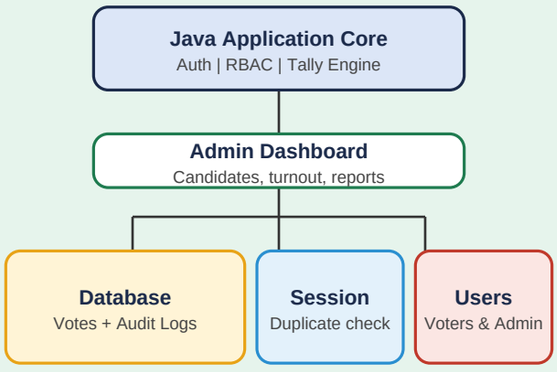
A Secure and Transparent Digital Voting Platform

ENGINEERING CAPSTONE PROJECT - 1 (231E4053R)

Abstract

The **Student Council Election Tally System** is a secure, efficient and fully automated digital voting platform that replaces error-prone, paper-based voting. It features **student authentication** via unique institutional IDs, **Role-Based Access Control (RBAC)** separating administrators from voters, **real-time automated vote counting**, **duplicate vote prevention** using session tracking, and **comprehensive audit logs**. An administrative dashboard lets officials manage candidates, monitor turnout live and generate instant, verifiable result reports, reducing processing time and fostering greater student participation.

System Architecture



MODULE-WISE IMPLEMENTATION

MODULE 1
From Problem to Program: Computational Thinking & Java Foundations

The election is modelled with **problem solving, algorithms and flowcharts** (register, authenticate, vote, tally). The **Java environment and program structure**, primitive data types, variables, constants, identifiers and literals store voter IDs, candidate codes and vote counts. **Operators** (arithmetic, relational, logical, assignment, ternary, unary, bitwise), typecasting and precedence/associativity drive calculations, while **console I/O** with formatted printing and **Scanner** reads typed input.

MODULE 2
Control Flow: Selection & Iteration

Conditional statements (if, if-else, nested-if, else-if ladder, switch with default and break) validate voter eligibility and route each ballot to the chosen candidate. **Loops** (while, do-while, for, nested loops) with continue and infinite loops keep the voting session running, iterate over all votes during tallying and stop when the election closes.

MODULE 3
Decomposition with Methods, Recursion & Arrays

Modular programming splits the system into methods with parameter passing, return values and overloading, such as castVote() and showResults(). **Recursion** uses base and recursive cases. **1D arrays** store votes per candidate (traversal, searching, summation, average, counting) and **2D arrays** hold matrix-style turnout tables.

MODULE 4
Object-Oriented Modelling, Design & Exception Handling

Classes and objects (Voter, Candidate, Admin, Election) use fields, methods, constructors (default, parameterized, overloaded, this), encapsulation and toString(); static members hold constants. **Inheritance, overriding and interfaces** give RBAC roles from a common User type. **Exception handling** (try-catch-finally, multi-catch, throw, throws) handles invalid IDs and duplicate votes.

MODULE 5
Strings, Library Algorithms & File I/O

String operations (indexing, substring, comparison with equals() vs ==, case conversion, searching) validate IDs, and **StringBuilder/StringBuffer** build report text. split(), join() and trim() parse records. **Arrays.sort(), Collections.sort(), binarySearch()** with Comparable/Comparator rank candidates, and **java.nio.file Files** read and write voter lists, results and audit logs.

Conclusion & Future Scope

This project applies core Java concepts, from problem solving and control flow to OOP, exception handling and file I/O, to build a secure and transparent digital voting platform. In future it can be extended with a GUI/web front-end, database storage, OTP or biometric verification and blockchain-backed audit trails.

PROJECT TEAM

N. Arjun Kireeti Roll No: 2620040117

K. Vishwaksena Reddy Roll No: 2620080044

GUIDE

Balaiah Gudipudi

Dept. of CSE, KLH Bachupally